IFT2830 - Exercices de Préparation Examen Final

Practice Drills for Final Exam Preparation

Date de création: 27 avril 2026
Cours: IFT2830 - Sécurité des systèmes informatiques
Professeur: Abdeltouab Belbekkouche
Session: Hiver 2026 (H26)

Partie 1 – Vrai ou Faux

Répondre par **V** pour Vrai ou **F** pour Faux. Donner une brève explication pour justifier votre réponse.

Triade CIA & Principes de Sécurité

#	Énoncé	V/F	Justification
1	La confidentialité protège l'information contre la modification non autorisée.	F	La confidentialité protège contre la **divulgation** non autorisée. C'est l'**intégrité** qui protège contre la modification.
2	Un système de backup régulier contribue principalement à la disponibilité des données.	V	Les backups permettent de restaurer les données en cas de perte, garantissant ainsi leur disponibilité.
3	Une signature numérique assure la confidentialité d'un message.	F	La signature numérique assure l'**authenticité**, l'**intégrité** et la **non-répudiation**, mais PAS la confidentialité. Pour la confidentialité, il faut chiffrer le message.
4	Le principe de moindre privilège (Least Privilege) stipule qu'un utilisateur doit avoir uniquement les permissions minimales nécessaires pour accomplir sa tâche.	V	C'est la définition exacte du principe de moindre privilège, qui réduit la surface d'attaque.
5	La défense en profondeur consiste à utiliser un seul mécanisme de sécurité très robuste.	F	La défense en profondeur utilise **plusieurs couches** de sécurité (physique, réseau, application, données) pour que si une couche échoue, les autres protègent encore.

Analyse de Risque & Gouvernance

#	Énoncé	V/F	Justification
6	La formule du risque est R = P + I (Risque = Probabilité + Impact).	F	La formule correcte est **R = P × I** (multiplication, pas addition). Un risque avec haute probabilité ET haut impact est très critique.
7	Une menace est une faiblesse dans un système qui peut être exploitée.	F	C'est la définition d'une **vulnérabilité**. Une **menace** est la cause potentielle d'un incident (attaquant, malware, incendie).
8	Le CISO (Chief Information Security Officer) opère au niveau stratégique de la gouvernance.	V	Le CISO est responsable de la stratégie de sécurité à long terme, du budget, et de la politique générale (niveau stratégique).
9	L'acceptation du risque est toujours la meilleure stratégie de traitement du risque.	F	L'acceptation du risque n'est appropriée que pour les **risques faibles** ou lorsque le coût de mitigation dépasse le risque lui-même. Pour les risques élevés, il faut éviter, réduire ou transférer.
10	Un actif peut être tangible (serveur) ou intangible (réputation, données).	V	Les actifs incluent à la fois les ressources physiques (matériel, bâtiments) et immatérielles (données clients, propriété intellectuelle, réputation).

Cryptographie Fondamentale

#	Énoncé	V/F	Justification
11	Le principe de Kerckhoffs stipule que la sécurité d'un système cryptographique doit reposer uniquement sur le secret de la clé, pas sur le secret de l'algorithme.	V	C'est le principe fondamental de la cryptographie moderne : l'algorithme peut être public (peer-reviewed), seule la clé doit rester secrète.
12	Un algorithme de hachage est réversible : on peut retrouver le message original à partir du hash.	F	Les fonctions de hachage sont **unidirectionnelles** (one-way). Il est impossible de retrouver l'input à partir du hash.
13	MD5 est recommandé pour le stockage des mots de passe en 2026.	F	MD5 est **cassé** depuis longtemps (collisions possibles). Pour les mots de passe, utiliser **bcrypt** ou **Argon2** avec salt.
14	Un MAC (Message Authentication Code) fournit à la fois l'intégrité et l'authenticité, mais pas la confidentialité.	V	Le MAC utilise une clé secrète pour générer un tag qui prouve l'intégrité (non modifié) et l'authenticité (provient du détenteur de la clé), mais ne chiffre pas le message.

15	SHA-256 est un algorithme de chiffrement symétrique.	F	SHA-256 est un **algorithme de hachage**, pas de chiffrement. Il produit un digest de 256 bits qui ne peut pas être déchiffré.

Cryptographie Symétrique

#	Énoncé	V/F	Justification
16	DES est un algorithme de chiffrement sécuritaire et recommandé pour les nouvelles implémentations en 2026.	F	DES utilise une clé de seulement 56 bits et a été cassé par brute force en 1999. Il est **obsolète**. Utiliser AES à la place.
17	Le mode ECB (Electronic Codebook) est sécuritaire et recommandé pour le chiffrement de fichiers.	F	ECB est **INSÉCURE** car il chiffre chaque bloc identiquement (patterns leak). **JAMAIS utiliser ECB**. Utiliser CBC, CTR ou GCM.
18	AES-256 utilise des blocs de 256 bits.	F	AES-256 utilise une **clé de 256 bits** mais des **blocs de 128 bits**. La taille du bloc est toujours 128 bits pour AES (128/192/256 se réfère à la taille de clé).
19	GCM (Galois/Counter Mode) est un mode AEAD (Authenticated Encryption with Associated Data) qui fournit à la fois confidentialité et intégrité.	V	GCM combine le mode CTR (chiffrement) avec GMAC (authentification), offrant confidentialité + intégrité + authenticité en un seul mode. C'est le mode recommandé (TLS 1.3).
20	Un cipher de flux (stream cipher) traite les données bit par bit ou byte par byte, tandis qu'un cipher par bloc traite des blocs fixes (64, 128 bits).	V	C'est la distinction principale : stream cipher (RC4, ChaCha20) pour streaming, block cipher (AES, DES) pour données volumineuses.

Cryptographie Asymétrique

#	Énoncé	V/F	Justification
21	Dans un système de cryptographie asymétrique, la clé publique sert à chiffrer et la clé privée sert à déchiffrer.	V	C'est le principe de base : n'importe qui peut chiffrer avec la clé publique, mais seul le détenteur de la clé privée peut déchiffrer.
22	Une signature numérique prouve la confidentialité d'un message.	F	La signature prouve l'**authenticité**, l'**intégrité** et la **non-répudiation**, mais PAS la confidentialité. Le message est envoyé en clair (sauf si chiffré séparément).

23	ECC-256 offre approximativement le même niveau de sécurité que RSA-3072.	V	Les clés ECC sont beaucoup plus courtes pour une sécurité équivalente : ECC-256 ≈ RSA-3072 ≈ AES-128 (128-bit security level).
24	L'algorithme de Shor menace la sécurité de RSA et ECC face aux ordinateurs quantiques.	V	L'algorithme de Shor peut casser la factorisation (RSA) et le logarithme discret (ECC) sur un ordinateur quantique suffisamment puissant. C'est pourquoi on développe la cryptographie post-quantique (NTRUEncrypt, CRYSTALS-Kyber).
25	Dans un système hybride (RSA + AES), on chiffre les données volumineuses avec RSA car c'est plus rapide.	F	C'est l'**inverse** : on chiffre les données avec **AES** (rapide) et on chiffre la clé AES avec **RSA** (lent mais sécurise l'échange de clé). C'est le modèle TLS/HTTPS.

Certificats Numériques & PKI

#	Énoncé	V/F	Justification
26	Un certificat DV (Domain Validation) valide l'identité de l'organisation propriétaire du domaine.	F	DV valide uniquement la **propriété du domaine** (quelques minutes), pas l'identité de l'organisation. Pour valider l'organisation, il faut un certificat **OV** (Organization Validation) ou **EV** (Extended Validation).
27	Dans une PKI, l'Autorité de Certification (CA) signe les certificats numériques.	V	La CA est responsable de signer les certificats avec sa clé privée après que la RA (Registration Authority) ait vérifié l'identité du demandeur.
28	OCSP (Online Certificate Status Protocol) est plus rapide et respecte mieux la vie privée que les CRL (Certificate Revocation Lists).	F	OCSP est plus rapide (requête temps réel, ~1KB) que CRL (fichier complet, MB), mais il est **MOINS privé** car la CA voit chaque vérification. **OCSP stapling** résout ce problème.
29	PKCS #10 définit le format des Certificate Signing Requests (CSR).	V	PKCS #10 est le standard pour les CSR (demandes de certificat) envoyées à la CA pour obtenir un certificat signé.
30	Dans le modèle de confiance hiérarchique, il existe un root CA qui signe des intermediate CAs, qui signent les end-entity certificates.	V	C'est le modèle standard des PKI d'entreprise et Internet : Root CA (offline, sécurisé) → Intermediate CA → End-entity cert.

Partie 2 – Questions à Choix Multiples

Dans chacun des cas suivants, choisir la ou les bonnes réponses.

Section A: Sécurité Réseau & Protocoles

1) Parmi les protocoles suivants, lesquels sont orientés connexion avec garantie de livraison?

- a. UDP
- b. TCP
- c. ICMP
- d. SCTP

Réponse: b, d

2) IPSec en mode tunnel chiffre:

- a. Uniquement le payload (données)
- b. Uniquement l'en-tête IP
- c. Le payload ET l'en-tête IP original
- d. Rien du tout (mode tunnel ne chiffre pas)

Réponse: c

3) Le mécanisme AH (Authentication Header) d'IPSec fournit:

- a. Confidentialité
- b. Authentification
- c. Intégrité
- d. Non-répudiation

Réponse: b, c

4) Le mécanisme ESP (Encapsulating Security Payload) d'IPSec fournit:

- a. Confidentialité
- b. Authentification
- c. Intégrité
- d. Disponibilité

Réponse: a, b, c

5) NAT (Network Address Translation) est utilisé pour:

- a. Chiffrer le trafic réseau
- b. Cacher la topologie interne du réseau
- c. Traduire des adresses IP privées en adresses publiques
- d. Détecter les intrusions

Réponse: b, c

6) Un VPN peut être implémenté avec: (Choisir tout ce qui s'applique)

- a. IPSec
- b. SSL/TLS
- c. PPTP
- d. HTTP

Réponse: a, b, c

7) La commande `ping` utilise quel protocole?

- a. TCP
- b. UDP
- c. ICMP
- d. ARP

Réponse: c

8) Pour trouver l'adresse MAC associée à une adresse IP sur le réseau local, on utilise:

- a. DNS
- b. ARP
- c. DHCP
- d. RARP

Réponse: b

9) Le port TCP 443 est utilisé par défaut pour:

- a. HTTP
- b. HTTPS
- c. SSH
- d. FTPS

Réponse: b

10) Le port TCP 22 est utilisé par défaut pour:

- a. Telnet
- b. SSH
- c. FTP
- d. SMTP

Réponse: b

Section B: Pare-feu & Défense

11) Un pare-feu qui examine le contenu applicatif (layer 7) pour bloquer des attaques spécifiques utilise:

- a. Filtrage statique
- b. Filtrage dynamique
- c. Inspection applicative (Application-layer firewall)
- d. NAT

Réponse: c

12) Un pare-feu qui maintient l'état des connexions TCP pour autoriser uniquement les réponses légitimes utilise:

- a. Filtrage statique
- b. Filtrage dynamique (stateful inspection)
- c. NAT
- d. Proxy

Réponse: b

13) Un routeur filtrant (screening router) est généralement utilisé pour:

- a. Filtrage en entrée (inbound filtering)
- b. Filtrage en sortie (outbound filtering)
- c. Chiffrement du trafic
- d. Authentification des utilisateurs

Réponse: a, b

14) Un honeypot peut être utilisé pour: (Choisir tout ce qui s'applique)

- a. Leurrer les attaquants
- b. Collecter des informations sur les nouvelles attaques
- c. Distraire les attaquants des ressources légitimes
- d. Remplacer un pare-feu

Réponse: a, b, c

15) La zone DMZ (DeMilitarized Zone) est typiquement utilisée pour:

- a. Héberger des serveurs accessibles depuis Internet (Web, Mail, DNS)
- b. Héberger la base de données principale de l'entreprise
- c. Isoler les serveurs publics du réseau interne
- d. Stocker les backups

Réponse: a, c

16) Un IDS (Intrusion Detection System) diffère d'un IPS (Intrusion Prevention System) car:

- a. IDS détecte et alerte, IPS détecte et bloque
- b. IDS est plus rapide que IPS
- c. IDS est placé en ligne, IPS en mode passif
- d. IDS coûte plus cher que IPS

Réponse: a

17) La défense en profondeur (Defense-in-Depth) signifie:

- a. Utiliser un seul mécanisme de sécurité très robuste
- b. Utiliser plusieurs couches de sécurité indépendantes
- c. Placer tous les serveurs dans un bunker souterrain
- d. Chiffrer toutes les données trois fois

Réponse: b

18) Un pare-feu est un outil de sécurité nécessaire mais pas suffisant car:

- a. Il ne peut pas détecter les attaques internes
- b. Il ne peut pas chiffrer les données
- c. Il ne peut pas vérifier l'intégrité des fichiers
- d. Toutes ces réponses

Réponse: d

Section C: Attaques Réseau

19) Une attaque SYN flood exploite:

- a. La phase de handshake TCP à trois voies
- b. Le protocole UDP
- c. Le protocole ICMP
- d. Le protocole ARP

Réponse: a

20) Dans une attaque par amplification DNS, le facteur d'amplification typique est:

- a. 2x
- b. 10x
- c. 50-100x
- d. 1000x

Réponse: c

21) L'attaque d'amplification la plus puissante connue utilise:

- a. DNS

- b. NTP (monlist)
- c. Memcached
- d. SSDP

Réponse: c (facteur 51,000x)

22) Une attaque de réflexion (reflection) utilise:

- a. Des serveurs légitimes comme "miroirs" pour cacher l'origine
- b. IP spoofing pour diriger les réponses vers la victime
- c. Le protocole TCP uniquement
- d. a et b

Réponse: d

23) La différence principale entre DoS et DDoS est:

- a. DoS vient d'une source, DDoS vient de multiples sources (botnet)
- b. DoS est plus difficile à détecter que DDoS
- c. DoS est plus puissant que DDoS
- d. Il n'y a pas de différence

Réponse: a

24) Pour se défendre contre une attaque DDoS massive, on peut: (Choisir tout ce qui s'applique)

- a. Utiliser un scrubbing center (Cloudflare, Akamai)
- b. Over-provisioning de bande passante
- c. Rate limiting
- d. Débrancher Internet

Réponse: a, b, c

25) L'usurpation d'adresse IP (IP spoofing) est possible car:

- a. Le protocole IP ne vérifie pas l'adresse source
- b. Les routeurs ne filtrent pas les paquets
- c. TCP n'a pas de mécanisme d'authentification
- d. Toutes ces réponses

Réponse: d

Section D: Malware & Exploits

26) Quel est le but principal d'un malware?

- a. Nuire à un système informatique
- b. Voler des mots de passe

- c. Créer un botnet
- d. Toutes ces réponses peuvent être des buts

Réponse: d

27) Un buffer overflow (débordement de tampon) exploite:

- a. Une mauvaise validation d'entrée dans le code
- b. Des zones mémoires non protégées
- c. L'injection de code exécutable en mémoire
- d. Toutes ces réponses

Réponse: d

28) Un keylogger peut être:

- a. Un logiciel installé sur l'ordinateur
- b. Un dispositif matériel branché entre le clavier et l'ordinateur
- c. Détecté uniquement par un antivirus
- d. a et b

Réponse: d

29) Un cheval de Troie (Trojan) se distingue d'un virus car:

- a. Il ne se réplique pas automatiquement
- b. Il se présente comme un logiciel légitime
- c. Il nécessite l'action de l'utilisateur pour s'installer
- d. Toutes ces réponses

Réponse: d

30) Un rootkit se caractérise par:

- a. Sa capacité à se cacher du système d'exploitation
- b. Son niveau de privilège élevé (root/administrator)
- c. Sa difficulté de détection et suppression
- d. Toutes ces réponses

Réponse: d

31) Un exploit zero-day est:

- a. Une vulnérabilité découverte mais non corrigée
- b. Une vulnérabilité inconnue du vendeur
- c. Une vulnérabilité avec un patch disponible depuis 0 jours
- d. a et b

Réponse: d

32) Un logiciel antivirus devrait être mis à jour:

- a. Annuellement
- b. Mensuellement
- c. Quotidiennement (ou automatiquement)
- d. Jamais (une fois installé suffit)

Réponse: c

33) L'objectif principal d'un Adware est:

- a. Détruire le système
- b. Afficher de la publicité
- c. Déterminer les habitudes d'achat
- d. b et c

Réponse: d

Section E: Attaques & Cycle de Vie

34) La première phase du cycle de vie d'une attaque est:

- a. Scanning
- b. Reconnaissance
- c. Intrusion
- d. Exploitation

Réponse: b

35) Les outils OSINT (Open Source Intelligence) sont utilisés pendant la phase de:

- a. Reconnaissance
- b. Intrusion
- c. Persistence
- d. Cleanup

Réponse: a

36) L'outil `whois` permet de:

- a. Trouver l'adresse IP d'un domaine
- b. Trouver le propriétaire/responsable d'un domaine
- c. Scanner les ports ouverts
- d. Détecter les vulnérabilités

Réponse: b

37) Un attaquant qui maintient l'accès après l'exploitation initiale utilise des techniques de:

- a. Reconnaissance
- b. Privilege escalation
- c. Persistence
- d. Cleanup

Réponse: c

38) Les techniques de "Living off the Land" consistent à:

- a. Utiliser des outils légitimes du système (PowerShell, WMI) pour l'attaque
- b. Cacher des fichiers dans le système de fichiers
- c. Installer de nouveaux outils
- d. Voler des données agricoles

Réponse: a

39) Une attaque par force brute contre les mots de passe consiste à:

- a. Tester toutes les combinaisons possibles
- b. Utiliser un dictionnaire de mots communs
- c. Réutiliser des credentials volés
- d. a uniquement

Réponse: d (force brute = toutes combinaisons; dictionnaire = attaque par dictionnaire)

40) Les rainbow tables sont:

- a. Des tables de hashes précalculés pour cracker les mots de passe
- b. Inefficaces contre les mots de passe avec salt
- c. Plus rapides que le bruteforce
- d. Toutes ces réponses

Réponse: d

Partie 3 – Questions de Développement

Répondre de manière détaillée et structurée.

Question 1: Triade CIA

Expliquez la triade CIA (Confidentialité, Intégrité, Disponibilité) en donnant pour chaque propriété:

- a) Une définition claire
- b) Un exemple d'attaque qui viole cette propriété
- c) Un mécanisme de défense approprié

Réponse attendue:

Confidentialité:

- *Définition:* L'information est accessible uniquement aux personnes autorisées. Empêcher la divulgation non autorisée.
- *Exemple d'attaque:* Écoute réseau (sniffing), vol de données, accès non autorisé à une base de données
- *Mécanisme de défense:* Chiffrement (AES, RSA), contrôle d'accès (ACL), VPN, authentification forte (MFA)

Intégrité:

- *Définition:* L'information n'est pas modifiée de manière non autorisée. Les données restent complètes, exactes et fiables.
- *Exemple d'attaque:* Modification de données en transit (man-in-the-middle), corruption de fichiers, SQL injection
- *Mécanisme de défense:* Hachage (SHA-256), signatures numériques, MAC, checksums, contrôle de version

Disponibilité:

- *Définition:* L'information est accessible aux personnes autorisées quand elles en ont besoin. Le système reste opérationnel.
- *Exemple d'attaque:* DoS/DDoS, ransomware, destruction de matériel, panne système
- *Mécanisme de défense:* Redondance (RAID, clusters), sauvegardes (backups), DDoS mitigation, plan de continuité (BCP/DRP), over-provisioning

Question 2: Analyse de Risque

Une entreprise de e-commerce identifie une vulnérabilité SQL injection (CVE score 8.5) sur son site de paiement. Si exploitée, elle pourrait exposer 50,000 numéros de cartes de crédit.

a) **Calculez le risque selon la formule $R = P \times I$. Justifiez vos valeurs.**

Réponse:

```
R = P × I

P (Probabilité): 0.85 (85%)
  - CVE score 8.5 indique haute probabilité d'exploitation
  - SQL injection est une attaque commune (OWASP Top 10 #1)
  - Site de paiement = cible attractive pour attaquants

I (Impact): 9/10
  - Exposition de 50,000 cartes de crédit
  - Coûts: amendes PCI-DSS, poursuites, perte de réputation
  - Coût estimé: $200 par carte × 50,000 = $10M+

R = 0.85 × 9 = 7.65 (RISQUE CRITIQUE)
```

b) **Proposez deux stratégies de traitement du risque avec justification.**

Réponse:

Stratégie 1: Réduction (Mitigation) — RECOMMANDÉ

- *Action:* Corriger immédiatement la vulnérabilité avec parameterized queries (prepared statements)
- *Justification:* C'est un risque critique sur un système critique. La correction est technique, rapide, et élimine quasi-totalement la vulnérabilité
- *Délai:* Immédiat (patch d'urgence dans 24-48h)
- *Coût:* Faible (quelques heures de développement) vs risque de $10M+

Stratégie 2: Transfert (partiel)

- *Action:* Souscrire une cyberassurance pour couvrir les coûts de breach
- *Justification:* Protection financière en cas d'exploitation malgré les contrôles
- *Note:* NE REMPLACE PAS la correction technique, c'est un complément

Stratégie NON APPROPRIÉE: Acceptation

- Accepter un risque critique de 7.65 sur un système de paiement serait **négligence criminelle**

Question 3: IPSec — AH vs ESP

À quels besoins de sécurité répondent les mécanismes AH et ESP du protocole IPSec? Expliquez la différence entre mode transport et mode tunnel.

Réponse:

AH (Authentication Header):

- *Sécurité fournie:*
  - ✅ Authentification (prouve l'origine)

- ✅ Intégrité (détecte modification)
- ❌ PAS de confidentialité (données en clair)

- *Usage:* Quand la confidentialité n'est pas nécessaire (données publiques) mais l'intégrité est critique

ESP (Encapsulating Security Payload):

- *Sécurité fournie:*
  - ✅ Confidentialité (chiffrement)
  - ✅ Authentification
  - ✅ Intégrité
- *Usage:* Protection complète (c'est le plus utilisé en pratique)

Mode Transport vs Mode Tunnel:

Aspect	Mode Transport	Mode Tunnel
Ce qui est protégé	Payload uniquement	Payload + En-tête IP original
En-tête IP	Original visible	Nouveau en-tête IP ajouté
Usage	Host-to-host sur même réseau	Gateway-to-gateway (VPN site-to-site)
Confidentialité adressage	❌ Non (IP src/dst visible)	✅ Oui (IP original caché)
Overhead	Faible	Plus élevé (double en-tête)

Exemple:

- *Transport:* Employee laptop → Corporate server (même réseau VPN)
- *Tunnel:* Office A → Office B à travers Internet (cache topologie interne)

Question 4: Pare-feu Limitations

Un pare-feu est un outil de sécurité nécessaire, mais pas suffisant. Expliquez pourquoi en donnant trois raisons concrètes avec exemples.

Réponse:

Raison 1: Limitations fonctionnelles — Ne couvre pas tous les besoins de sécurité

Le pare-feu contribue au **contrôle d'accès** et à la **visibilité limitée** du réseau interne, mais ne peut pas:

- **Chiffrer les données:** Un pare-feu ne protège pas la confidentialité des données en transit. Il faut utiliser TLS/HTTPS, VPN, ou chiffrement disque.
- **Vérifier l'intégrité:** Il ne détecte pas la corruption de fichiers ou la modification de données. Il faut des checksums (SHA-256), signatures numériques, ou systèmes de contrôle de version.

- **Authentifier les utilisateurs:** Il filtre par IP/port, pas par identité utilisateur. Il faut IAM (Identity and Access Management), MFA, SSO.

Exemple: Un attaquant qui obtient un accès légitime via phishing peut exfiltrer des données non chiffrées. Le pare-feu voit du trafic HTTPS légitime et ne peut rien faire.

Raison 2: Point unique de défaillance (Single Point of Failure)

Un pare-feu unique crée un goulot d'étranglement:

- **Performance:** Tout le trafic passe par un point → dégradation possible
- **Disponibilité:** Si le pare-feu tombe en panne, le réseau entier peut être coupé
- **Contournement:** Si l'attaquant contourne le pare-feu (WiFi non sécurisé, VPN personnel, clé USB), toute la défense est inutile

Exemple: Employee qui branche un modem 4G sur son PC de bureau contourne complètement le pare-feu d'entreprise.

Raison 3: Faux sentiment de sécurité → Négligence des autres contrôles

Croire que "j'ai un pare-feu donc je suis protégé" est dangereux:

- **Attaques applicatives:** Pare-feu ne peut pas détecter SQL injection, XSS, CSRF dans du trafic HTTPS légitime
- **Attaques internes:** 30% des attaques viennent d'**insiders** (employés malveillants ou négligents) qui sont déjà derrière le pare-feu
- **Social engineering:** Phishing, hameçonnage ne sont pas bloqués par le pare-feu

Exemple: Attaquant qui envoie un email de phishing avec un lien vers un site malveillant. L'utilisateur clique, télécharge un trojan. Le pare-feu voit du trafic HTTPS sortant légitime.

Conclusion: Défense en profondeur requise
Un pare-feu doit être utilisé **en combinaison** avec:

- Antivirus/EDR
- IDS/IPS
- Chiffrement (VPN, TLS)
- Authentification forte (MFA)
- Formation utilisateurs (security awareness)
- Logs centralisés (SIEM)
- Backups
- Politique de sécurité cohérente

Question 5: Cryptographie Symétrique vs Asymétrique

Comparez la cryptographie symétrique et asymétrique sur les aspects suivants: vitesse, distribution de clés, usage typique. Donnez un exemple d'utilisation hybride.

Réponse:

Aspect	Symétrique	Asymétrique
Nombre de clés	1 clé (partagée)	2 clés (publique + privée)
Vitesse	✅ Très rapide (AES: plusieurs GB/s)	❌ Lent (RSA: ~100x plus lent)
Taille de clés	Petite (128-256 bits)	Grande (2048-4096 bits)
Distribution de clés	❌ Problème majeur (comment échanger la clé secrète?)	✅ Simple (clé publique peut être distribuée ouvertement)
Scalabilité	❌ Mauvaise (N utilisateurs = N(N-1)/2 clés)	✅ Bonne (N utilisateurs = N paires de clés)
Usage typique	Chiffrement de données volumineuses (fichiers, disques, communications)	Échange de clés, signatures numériques, certificats
Exemples	AES-256, ChaCha20, 3DES	RSA-2048, ECC-256, Ed25519

Chiffrement Hybride: RSA + AES (modèle TLS/HTTPS)

Problème résolu: On veut la **sécurité de RSA** (échange de clé sécurisé) avec la **vitesse de AES** (chiffrement bulk data).

Processus:

1. **Génération clé session (Alice):**

```
Alice génère une clé AES-256 aléatoire (32 bytes)
Clé AES = [random 256 bits]
```

2. **Chiffrement de la clé (Alice):**

```
Alice récupère la clé publique RSA de Bob
Clé_chiffrée = RSA_encrypt(Clé_AES, PublicKey_Bob)
```

3. **Chiffrement des données (Alice):**

```
Alice chiffre le message avec AES (rapide)
Message_chiffré = AES-256-GCM(Message, Clé_AES)
```

4. **Envoi (Alice → Bob):**

```
Alice envoie: [Clé_chiffrée, Message_chiffré]
```

5. **Déchiffrement de la clé (Bob):**

```
Bob déchiffre la clé AES avec sa clé privée RSA
Clé_AES = RSA_decrypt(Clé_chiffrée, PrivateKey_Bob)
```

6. **Déchiffrement des données (Bob):**

```
Bob déchiffre le message avec AES (rapide)
Message = AES-256-GCM_decrypt(Message_chiffré, Clé_AES)
```

Avantages:

- ✅ Sécurité: Échange de clé protégé par RSA
- ✅ Performance: Bulk data chiffré par AES (rapide)
- ✅ Scalabilité: Chaque session utilise une nouvelle clé AES (forward secrecy)

Exemple réel: TLS 1.2/1.3 (HTTPS), PGP/GPG, S/MIME

Question 6: Digital Signatures — 5 Steps

Expliquez le processus complet de signature numérique en 5 étapes. Quelles propriétés de sécurité sont garanties? Qu'est-ce qui n'est PAS garanti?

Réponse:

Processus de Signature Numérique:

Étape 1: Résumé (Hashing) — Expéditeur

```
Alice veut signer un message M
Hash = SHA-256(M)
Exemple: M = "Contrat validé" → Hash = a3f5b8c...
```

Étape 2: Chiffrement du résumé (Signing) — Expéditeur

```
Alice chiffre le hash avec sa clé privée
Signature = Hash^d mod n  (RSA)
OU: Signature = Sign(Hash, PrivateKey_Alice)  (notation générique)
```

Étape 3: Envoi (Message + Signature) — Expéditeur

```
Alice envoie à Bob:
  - Message original M (en clair)
  - Signature S
```

Étape 4: Résumé du message reçu (Hashing) — Destinataire

```
Bob reçoit M et S
Bob calcule le hash du message reçu:
Hash_received = SHA-256(M)
```

Étape 5: Vérification (Verification) — Destinataire

```
Bob déchiffre la signature avec la clé publique d'Alice:
Hash_decrypted = Signature^e mod n  (RSA)
OU: Hash_decrypted = Verify(Signature, PublicKey_Alice)

Si Hash_received == Hash_decrypted:
  ✓ Signature VALIDE
  → Message authentique, intègre, non-répudiation
Sinon:
  ✗ Signature INVALIDE
  → Message modifié OU signature forgée OU mauvaise clé
```

Propriétés de Sécurité GARANTIES:

Propriété	Explication
✅ **Authenticité**	Prouve que le message vient du détenteur de la clé privée (Alice)
✅ **Intégrité**	Détecte toute modification du message (hash change)
✅ **Non-répudiation**	Alice ne peut pas nier avoir signé (seule elle possède la clé privée)

Propriété NON GARANTIE:

Propriété	Explication
❌ **Confidentialité**	Le message M est envoyé **en clair**. N'importe qui peut le lire. Pour la confidentialité, il faut **chiffrer le message séparément** (ex: avec la clé publique du destinataire).

Exemple Concret:

```
Message: "Transfert 10,000€ au compte XYZ"
Signature: [proves Alice signed this]

Attaquant peut:
  ❌ Lire le message (c'est en clair)
  ❌ Modifier le message (signature devient invalide → détecté)
  ❌ Forger une signature (n'a pas la clé privée d'Alice)

Pour confidentialité + signature:
  1. Signer le message (signature numérique)
  2. Chiffrer message+signature avec clé publique de Bob
```

Question 7: RSA Key Generation

Générez une paire de clés RSA avec p=11 et q=13. Montrez toutes les étapes de calcul. Utilisez e=7. Chiffrez puis déchiffrez M=5.

Réponse:

Étape 1: Calculer n

```
n = p × q = 11 × 13 = 143
```

Étape 2: Calculer ϕ(n) (Euler's totient)

```
φ(n) = (p - 1) × (q - 1)
φ(n) = (11 - 1) × (13 - 1)
φ(n) = 10 × 12 = 120
```

Étape 3: Choisir e

```
e = 7  (donné)

Vérifier: gcd(e, φ(n)) = gcd(7, 120) = 1 ✓
(7 et 120 sont premiers entre eux)
```

Étape 4: Calculer d (exposant privé)

```
Trouver d tel que: (e × d) mod φ(n) = 1
(7 × d) mod 120 = 1

Méthode: Algorithme d'Euclide étendu
7 × d ≡ 1 (mod 120)

Test:
7 × 103 = 721 = 6 × 120 + 1 → 721 mod 120 = 1 ✓

d = 103
```

Clés générées:

```
Clé publique:  (n, e) = (143, 7)
Clé privée:    (n, d) = (143, 103)

IMPORTANT: Détruire p=11, q=13, φ(n)=120
```

Chiffrement de M=5 (avec clé publique):

```
C = M^e mod n
C = 5^7 mod 143

Calcul:
5^1 = 5
5^2 = 25
5^4 = 625 mod 143 = 625 - 4×143 = 625 - 572 = 53
5^7 = 5^4 × 5^2 × 5^1 = 53 × 25 × 5 mod 143

53 × 25 = 1325 mod 143 = 1325 - 9×143 = 1325 - 1287 = 38
38 × 5 = 190 mod 143 = 190 - 143 = 47

C = 47
```

Déchiffrement de C=47 (avec clé privée):

```
M = C^d mod n
M = 47^103 mod 143

Calcul (exponentiation rapide):
47^1 = 47
47^2 = 2209 mod 143 = 2209 - 15×143 = 64
47^4 = 64^2 = 4096 mod 143 = 4096 - 28×143 = 92
47^8 = 92^2 = 8464 mod 143 = 8464 - 59×143 = 127
47^16 = 127^2 = 16129 mod 143 = 16129 - 112×143 = 113
47^32 = 113^2 = 12769 mod 143 = 12769 - 89×143 = 42
47^64 = 42^2 = 1764 mod 143 = 1764 - 12×143 = 48

103 = 64 + 32 + 4 + 2 + 1 (binaire: 1100111)

47^103 = 47^64 × 47^32 × 47^4 × 47^2 × 47^1 mod 143
       = 48 × 42 × 92 × 64 × 47 mod 143

Calcul progressif:
48 × 42 = 2016 mod 143 = 13
13 × 92 = 1196 mod 143 = 68
68 × 64 = 4352 mod 143 = 71
71 × 47 = 3337 mod 143 = 5

M = 5 ✓
```

Vérification: Message original M=5 retrouvé après chiffrement/déchiffrement.

Question 8: Certificate Validation Chain

Expliquez le processus complet de validation d'un certificat SSL/TLS par un navigateur web. Quels sont les points de vérification?

Réponse:

Processus de Validation de Certificat (Navigateur Web):

Étape 1: Récupération du certificat

```
Utilisateur: https://example.com
Navigateur envoie: ClientHello
Serveur répond: ServerHello + Certificate
Navigateur reçoit: Certificat du serveur (end-entity certificate)
```

Étape 2: Vérification de la chaîne de certificats

```
Certificat contient:
  - End-entity cert (example.com)
  - Intermediate CA cert(s)
  - (Root CA est dans le trust store du navigateur)

Navigateur reconstruit la chaîne:
Root CA → Intermediate CA → End-entity (example.com)
```

Étape 3: Vérification de la signature (chaque niveau)

```
Pour chaque certificat dans la chaîne:

1. Extraire la clé publique du certificat parent
2. Vérifier la signature du certificat enfant:
   - Calculer hash du certificat enfant
   - Déchiffrer signature avec clé publique du parent
   - Comparer hash calculé vs hash déchiffré

Si signature invalide → ERREUR "Certificate not trusted"
```

Étape 4: Vérification du Root CA

```
Navigateur vérifie:
  - Le Root CA est-il dans le trust store?
  - Trust store = liste des CAs de confiance (Mozilla, Microsoft, Apple)

Si Root CA inconnu → ERREUR "Unknown Certificate Authority"
```

Étape 5: Vérification de la validité temporelle

```
Pour chaque certificat:
  - Not Before <= Date actuelle <= Not After

Exemple:
  Not Before: 2025-01-01 00:00:00 UTC
  Not After:  2026-01-01 00:00:00 UTC
  Date actuelle: 2026-04-27 → DANS LA PÉRIODE ✓

Si expiré → ERREUR "Certificate expired"
Si pas encore valide → ERREUR "Certificate not yet valid"
```

Étape 6: Vérification du nom de domaine (Subject/SAN)

```
Navigateur vérifie que le certificat est émis pour le bon domaine:

Champs vérifiés:
  1. Common Name (CN): CN=example.com
  2. Subject Alternative Name (SAN): DNS:example.com, DNS:www.example.com

Wildcard accepté: *.example.com couvre www.example.com, api.example.com
Mais PAS: *.*.example.com ou example.com seul

Si mismatch → ERREUR "Certificate name mismatch"
```

Étape 7: Vérification de révocation

```
Navigateur vérifie si le certificat a été révoqué:

Méthode 1: CRL (Certificate Revocation List)
  - Télécharger CRL depuis l'URL dans le certificat
  - Chercher le numéro de série dans la liste
  - Problème: Fichier lourd (MB), mise à jour lente (24-48h)

Méthode 2: OCSP (Online Certificate Status Protocol)
  - Envoyer requête OCSP au serveur OCSP de la CA
  - Recevoir réponse: Good / Revoked / Unknown
  - Problème: Latence, privacy (CA voit chaque requête)

Méthode 3: OCSP Stapling (recommandé)
  - Serveur cache la réponse OCSP et l'envoie avec le certificat
  - Pas de requête supplémentaire, plus rapide, meilleur privacy

Si révoqué → ERREUR "Certificate revoked"
```

Étape 8: Vérification des extensions (Key Usage, EKU)

```
Key Usage: Certificat peut-il être utilisé pour l'usage prévu?
  - Key Usage: Digital Signature, Key Encipherment
  - Extended Key Usage (EKU): Server Authentication (TLS Web Server)

Si usage inapproprié → ERREUR "Certificate usage violation"
```

Étape 9: Vérification des contraintes (Basic Constraints)

```
Pour les CA certificates:
  - Basic Constraints: CA=TRUE
  - Path Length Constraint: max 1 niveau d'intermediate CA

Si violation → ERREUR "Path length constraint exceeded"
```

Étape 10: Validation réussie → Établissement de la connexion

```
Si toutes les vérifications passent:
  ✓ Certificat VALIDE
  → Navigateur affiche le cadenas vert/gris
  → Établissement de la session TLS
  → Échange de clés (DHE/ECDHE)
  → Communication chiffrée (AES-GCM)
```

Points de Vérification Résumé:

#	Vérification	Échec → Erreur
1	Chaîne de certificats complète	Chain incomplete
2	Signatures valides (chaque niveau)	Signature invalid
3	Root CA de confiance	Unknown CA
4	Dates de validité (Not Before/After)	Expired / Not yet valid
5	Nom de domaine (CN/SAN match)	Name mismatch
6	Révocation (CRL/OCSP)	Revoked
7	Key Usage / EKU	Usage violation
8	Basic Constraints (CA path)	Path constraint

Exemple d'Erreur Courante:

```
Site: https://example.com
Erreur: "NET::ERR_CERT_DATE_INVALID"

Cause:
  Certificate Not After: 2024-01-01 (expiré depuis 1 an)
  Date actuelle: 2026-04-27

Solution (admin site):
  - Renouveler le certificat auprès de la CA
  - Installer le nouveau certificat sur le serveur
  - Configurer renouvellement automatique (Let's Encrypt)
```

Question 9: SQL Injection Attack & Defense

a) Expliquez ce qu'est une injection SQL avec un exemple de code vulnérable.
b) Montrez comment un attaquant peut exploiter cette vulnérabilité.
c) Proposez trois méthodes de défense, en expliquant laquelle est la meilleure.

Réponse:

a) Définition et Code Vulnérable

Injection SQL: Technique d'attaque qui consiste à insérer du code SQL malveillant dans une requête pour modifier son comportement et accéder/modifier/supprimer des données non autorisées.

Code Vulnérable (Python + SQLite):

```
# ❌ VULNÉRABLE - String concatenation
import sqlite3

def login(username, password):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # Concaténation de strings = DANGER
    query = "SELECT * FROM users WHERE username = '" + username + "' AND password
= '" + password + "'"

    cursor.execute(query)
    user = cursor.fetchone()

    if user:
        return "Login successful"
    else:
        return "Invalid credentials"
```

b) Exploitation par un Attaquant

Attaque 1: Authentication Bypass

```
# Attaquant entre:
username = "admin' OR '1'='1"
password = "anything"

# Requête construite:
SELECT * FROM users WHERE username = 'admin' OR '1'='1' AND password = 'anything'

# Évaluation:
WHERE username = 'admin'  → False (probablement)
OR '1'='1'                → True (toujours vrai)
AND password = 'anything' → True pour la clause OR

# Résultat: L'attaquant se connecte sans connaître le mot de passe!
```

Attaque 2: Data Exfiltration (Union-based)

```
# Attaquant entre:
username = "admin' UNION SELECT credit_card, cvv, expiry FROM payment_info--"
password = ""

# Requête construite:
SELECT * FROM users WHERE username = 'admin'
UNION SELECT credit_card, cvv, expiry FROM payment_info--' AND password = ''

# -- commente le reste de la requête
# UNION combine les résultats de deux tables
# Résultat: Attaquant récupère toutes les cartes de crédit
```

Attaque 3: Blind SQL Injection (Time-based)

```
# Attaquant entre:
username = "admin' AND (SELECT CASE WHEN (1=1) THEN pg_sleep(5) ELSE 0 END)--"

# Si la réponse prend 5 secondes → condition vraie
# Si réponse immédiate → condition fausse
# Attaquant peut extraire données bit par bit
```

Attaque 4: Destructive (DROP TABLE)

```
# Attaquant entre:
username = "admin'; DROP TABLE users;--"

# Requêtes exécutées:
SELECT * FROM users WHERE username = 'admin';
DROP TABLE users;--' AND password = ''

# Résultat: Table users SUPPRIMÉE!
```

c) Méthodes de Défense

Défense 1: Parameterized Queries (Prepared Statements) — ✅ MEILLEURE

```
# ✅ SÉCURISÉ - Parameterized query
def login_secure(username, password):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    # ? est un placeholder, pas une concaténation
    query = "SELECT * FROM users WHERE username = ? AND password = ?"

    # Parameters sont passés séparément
    cursor.execute(query, (username, password))
    user = cursor.fetchone()

    if user:
        return "Login successful"
    else:
        return "Invalid credentials"

# Si attaquant entre: username = "admin' OR '1'='1"
# La base de données traite TOUTE la chaîne comme un literal string:
# WHERE username = "admin' OR '1'='1"  (cherche EXACTEMENT ce username)
# → Aucun utilisateur trouvé → Login échoue ✓
```

Pourquoi c'est la meilleure:

- ✅ La base de données distingue le code SQL des données
- ✅ Les caractères spéciaux sont automatiquement échappés
- ✅ Fonctionne pour tous les types d'injection SQL
- ✅ Performance: requête compilée une fois, réutilisée
- ✅ Aucune maintenance: pas de liste de caractères à filter

Défense 2: ORM (Object-Relational Mapping)

```python
# ✅ BON - SQLAlchemy ORM
from sqlalchemy.orm import sessionmaker
from models import User

def login_orm(username, password):
    session = Session()

    # ORM utilise parameterized queries en interne
    user = session.query(User).filter(
        User.username == username,
        User.password == password
    ).first()

    if user:
        return "Login successful"
    else:
        return "Invalid credentials"
```

Avantages:

- ✅ Abstraction: pas de SQL direct
- ✅ Sécurisé par défaut (utilise prepared statements)

Limites:

- ⚠️ Requêtes raw SQL dans l'ORM peuvent encore être vulnérables
- ⚠️ Performance: overhead de l'ORM

Défense 3: Input Validation + Escaping

```
# ⚠️ INSUFFISANT - Input validation seule
import re

def login_validated(username, password):
    # Whitelist: accepter seulement alphanumeric
    if not re.match(r'^[a-zA-Z0-9_]+$', username):
        return "Invalid username format"

    if not re.match(r'^[a-zA-Z0-9@!#$%]+$', password):
        return "Invalid password format"

    # Toujours utiliser parameterized query!
    query = "SELECT * FROM users WHERE username = ? AND password = ?"
    cursor.execute(query, (username, password))
    # ...
```

Pourquoi INSUFFISANT seul:

- ❌ Filtrer les caractères spéciaux est incomplet (il y a toujours des bypasses)
- ❌ Ne protège pas contre toutes les formes d'injection
- ❌ Peut bloquer des inputs légitimes (ex: nom avec apostrophe: O'Brien)
- ✅ Utile en **complément** (defense-in-depth), pas comme défense primaire

Comparaison:

Défense	Efficacité	Complexité	Recommandation
Parameterized Queries	✅ 100%	Faible	**TOUJOURS utiliser**
ORM	✅ 95% (si bien utilisé)	Moyenne	Bon, mais attention aux raw queries
Input Validation	⚠️ 30% (insuffisant seul)	Moyenne	Complément uniquement
WAF	⚠️ 60% (bypassable)	Haute	Defense-in-depth

Défense Additionnelle (Defense-in-Depth):

- **Least Privilege:** User de base de données avec permissions minimales (SELECT only sur tables nécessaires, pas de DROP, DELETE)
- **Error Handling:** Ne pas exposer les erreurs SQL détaillées à l'utilisateur
- **Logging:** Logger toutes les requêtes SQL suspectes

- **WAF:** Web Application Firewall pour détecter patterns SQLi
- **Regular Audits:** Scan de code (SAST) et pentesting

Question 10: Phishing Attack Scenario

Une compagnie propose un service web où les clients se connectent avec username/password pour gérer leurs fichiers. Décrivez:

a) Comment un attaquant pourrait mener une attaque de phishing (hameçonnage) contre les clients.
b) Pourquoi un pare-feu ne protège pas contre ce type d'attaque.
c) Trois mesures de défense efficaces.

Réponse:

a) Scénario d'Attaque de Phishing

Étape 1: Reconnaissance

```
Attaquant:
- Visite le site légitime: https://happyendings.com
- Copie le HTML/CSS complet (clone parfait)
- Enregistre un domaine similaire:
  - happyending.com (sans 's')
  - happy-endings.com (avec tiret)
  - happyendings.net (différent TLD)
```

Étape 2: Setup Infrastructure

```
Attaquant configure:
- Serveur web avec le clone du site
- Certificat SSL (Let's Encrypt = gratuit, légitime)
- Page de login identique pixel par pixel
- Script backend pour capturer credentials:

  login.php:
    - Récupérer username + password
    - Stocker dans base de données attaquant
    - Rediriger vers vrai site (ou afficher "erreur temporaire")
```

Étape 3: Campagne de Phishing

```
Attaquant envoie des emails aux clients:

From: support@happyendings.com (spoofé)
Subject: URGENT: Votre document est prêt!
Body:
  "Bonjour,

  Votre script adapté est maintenant disponible.
  Veuillez vous connecter pour le télécharger:

  https://happy-endings.com/login

  Ce lien expire dans 24 heures.

  Cordialement,
  L'équipe Happy Endings Inc."
```

Étape 4: Exploitation

```
Victime:
1. Reçoit l'email (semble légitime)
2. Clique sur le lien → happy-endings.com (phishing site)
3. Voit page de login (identique à l'original)
4. Entre username: "client123"
5. Entre password: "P@ssw0rd!"
6. Clique "Login"

Attaquant:
- Capture username + password
- Stocke: [client123, P@ssw0rd!]
- Redirige vers vrai site (ou affiche "erreur serveur")

Victime ne suspecte rien si redirection bien faite.
```

Étape 5: Utilisation des Credentials

```
Attaquant:
- Se connecte au VRAI site: https://happyendings.com
- Username: client123
- Password: P@ssw0rd!
- Accède à tous les fichiers du client
- Peut voler propriété intellectuelle (scripts)
- Peut uploader malware
```

b) Pourquoi le Pare-feu Ne Protège Pas

Raison 1: Protocole Légitime

```
Trafic vu par le pare-feu:

Victime → happy-endings.com (phishing)
  - Protocol: HTTPS (port 443)
  - Méthode: POST /login
  - Payload: chiffré par TLS

Pare-feu voit:
  ✓ HTTPS valide (port 443 autorisé)
  ✓ Certificat SSL valide (Let's Encrypt)
  ✓ Trafic sortant légitime

→ Pare-feu ne peut PAS bloquer
```

Raison 2: Niveau Applicatif

```
Phishing = attaque Layer 7 (Application)
Pare-feu = filtre Layer 3-4 (Network/Transport)

Pare-feu filtre sur:
  - Adresse IP source/destination
  - Port source/destination
  - Protocol (TCP/UDP)

Pare-feu NE PEUT PAS:
  - Lire le contenu d'un email
  - Analyser le contenu d'une page HTTPS (chiffré)
  - Distinguer happy-endings.com (phishing) de happyendings.com (légitime)
  - Détecter l'intention malveillante d'un site
```

Raison 3: Attaque Sociale, Pas Technique

```
Le pare-feu protège contre:
  ✅ Scans de ports
  ✅ Attaques DoS/DDoS
  ✅ Trafic non autorisé

Le pare-feu NE protège PAS contre:
  ❌ Ingénierie sociale (manipulation humaine)
  ❌ Utilisateur qui donne volontairement son mot de passe
  ❌ Contenu malveillant dans protocoles autorisés (HTTPS, email)
```

Quand l'attaquant utilise les credentials volés:

```
Attaquant → happyendings.com (vrai site)
  - Connexion avec credentials légitimes
  - Trafic HTTPS normal
  - Pare-feu voit: trafic légitime ✓

→ Pare-feu ne peut PAS détecter que ce n'est pas le vrai client
```

c) Trois Mesures de Défense Efficaces

Défense 1: Authentification Multi-Facteur (MFA) — ✅ PLUS EFFICACE

```
Implémentation:
  1. User entre username + password
  2. Système envoie code unique (6 chiffres):
     - Via SMS: "Code: 123456"
     - Via app (Google Authenticator, Authy)
     - Via email (moins sécurisé)
  3. User doit entrer le code dans 5 minutes
  4. Code vérifié par serveur
  5. Accès accordé SI code correct

Pourquoi ça marche contre phishing:
  ✓ Attaquant capture username + password
  ✗ Mais n'a PAS le code MFA (envoyé au téléphone du vrai user)
  → Ne peut pas se connecter

Types de MFA:
  - SMS (acceptable, mais SIM swapping possible)
  - TOTP (Time-based One-Time Password) - Google Authenticator (MEILLEUR)
  - Hardware token (YubiKey) - BEST (résiste au phishing avancé)
  - Biometric (fingerprint, face) - Si combiné avec device
```

Défense 2: Security Awareness Training (Formation des Utilisateurs)

```
Programme de formation:

  1. Email de phishing simulés (exercices):
     - Envoyer faux phishing aux employés
     - Tracker qui clique
     - Formation immédiate pour ceux qui tombent

  2. Enseignement des signes de phishing:
     ⚠️ Urgence artificielle ("expire dans 24h")
     ⚠️ Fautes d'orthographe/grammaire
     ⚠️ Domaine suspect (happy-endings.com vs happyendings.com)
     ⚠️ Demande de credentials par email/SMS
     ⚠️ Lien différent du texte affiché

  3. Pratiques sécuritaires:
     ✅ Toujours taper l'URL manuellement (pas cliquer liens)
     ✅ Vérifier certificat SSL (cadenas, nom domaine)
     ✅ Activer MFA partout
     ✅ Utiliser gestionnaire de mots de passe (remplit uniquement sur bon
domaine)
     ✅ Signaler emails suspects (bouton "Report Phishing")

  4. Mesures:
     - Taux de clic sur phishing simulés < 5%
     - Taux de signalement > 70%
```

Défense 3: Monitoring & Detection

```
1. Breach Monitoring:
   - Service: Have I Been Pwned (HIBP), SpyCloud
   - Vérifier si credentials ont été leakés
   - Forcer reset password si détecté

2. Anomaly Detection:
   - Location: User se connecte normalement de Montréal
     → Connexion depuis Russie 5 min plus tard → ALERTE
   - Device: User utilise toujours Chrome/Windows
     → Connexion depuis Firefox/Linux → ALERTE
   - Behavior: User télécharge 1-2 fichiers/jour
     → Téléchargement de 100 fichiers en 10 min → ALERTE

3. Rate Limiting:
   - Max 5 tentatives de login par compte par heure
   - Après 3 échecs: CAPTCHA
   - Après 5 échecs: Blocage temporaire + email alerte

4. Session Management:
   - Session timeout: 30 minutes d'inactivité
   - Invalidation immédiate si:
     - Changement d'IP
     - Changement de user-agent
     - Détection de compromission
```

Défenses Additionnelles (Defense-in-Depth):

```
4. Email Security (Anti-Phishing):
   - DMARC/SPF/DKIM (empêche spoofing du domaine)
   - Email gateway (Proofpoint, Mimecast)
   - Link protection (réécriture URLs, scan en temps réel)
   - Banner warnings ("Email externe")

5. Browser Protection:
   - Google Safe Browsing (liste noire de sites phishing)
   - Enterprise browser policy (bloquer sites non whitelisted)
   - Extension anti-phishing (Netcraft, PhishTank)

6. Certificate Monitoring:
   - Certificate Transparency logs
   - Alertes si nouveau cert émis pour domaine similaire
   - Détection de typosquatting

7. Incident Response:
   - Si phishing détecté:
     1. Bloquer domaine phishing (firewall, DNS)
     2. Notifier tous les clients (email + SMS)
     3. Forcer reset password pour comptes compromis
     4. Contacter autorités (signaler domaine)
     5. Post-mortem: améliorer détection
```

Comparaison:

Défense	Efficacité vs Phishing	Coût	Complexité
MFA	✅ 99% (si hardware token)	Faible	Faible
Security Training	✅ 70-80% (si continu)	Moyen	Moyenne
Monitoring	✅ 60% (détection post-breach)	Élevé	Élevée
Email Security	✅ 50-70% (pas 100%)	Élevé	Moyenne
Pare-feu	❌ 0%	N/A	N/A

Conclusion: MFA est la **défense la plus efficace** contre phishing (bloque même si user donne password). Mais **defense-in-depth** recommandée (MFA + Training + Monitoring).

Question 11: DoS vs DDoS

Expliquez la différence entre DoS et DDoS. Décrivez trois types d'attaques DDoS avec facteurs d'amplification. Proposez des stratégies de mitigation.

Réponse:

DoS (Denial of Service) vs DDoS (Distributed Denial of Service):

Aspect	DoS	DDoS
Sources	Une seule machine	Multiple machines (botnet: 10,000-100,000+)
Volume	Limité (bande passante attaquant: ~100 Mbps)	Massif (agrégat botnet: 1-10 Tbps)
Difficulté attaque	Facile (script kiddie, LOIC)	Complex (C&C infrastructure, malware)
Détection	Facile (bloquer IP source)	Difficile (IPs distribuées géographiquement)
Mitigation	Simple (firewall rule: block IP)	Nécessite DDoS mitigation service (Cloudflare, Akamai)
Coût	Presque gratuit	Infrastructure coûteuse (sauf botnet loué)
Exemple	Ping of Death, SYN flood depuis 1 IP	Mirai botnet (IoT devices, 600 Gbps)

Trois Types d'Attaques DDoS avec Amplification:

Attaque 1: DNS Amplification

```
Facteur d'amplification: 50x - 100x

Mécanisme:
  1. Attaquant envoie requête DNS:
     - Source IP: SPOOFÉE = IP de la victime
     - Destination: Serveur DNS récursif ouvert
     - Query: ANY record pour un domaine (retourne tous records)
     - Taille requête: ~60 bytes

  2. Serveur DNS répond:
     - Destination: IP victime (IP spoofée)
     - Réponse: Tous les records (A, AAAA, MX, TXT, NS, etc.)
     - Taille réponse: ~3,000-6,000 bytes

  3. Amplification:
     Requête 60 bytes → Réponse 3,000 bytes = 50x

  4. Attaque massive:
     - Attaquant contrôle botnet de 10,000 machines
     - Chaque bot envoie 100 requêtes/sec
     - 10,000 × 100 × 60 bytes = 60 MB/s OUT
     - 60 MB/s × 50 = 3 GB/s vers VICTIME
     - Victime reçoit 3 Gbps de trafic DNS non sollicité

Défense:
  - DNS récursif: Désactiver récursion pour requêtes externes
  - DNS récursif: Rate limiting par IP source
  - Victime: Anycast (distribuer trafic géographiquement)
  - Victime: Scrubbing center (filtrer trafic DNS)
```

Attaque 2: NTP Amplification (monlist command)

```
Facteur d'amplification: 556x (!) [Record historique avant correctif]

Mécanisme:
  1. Attaquant envoie requête NTP:
     - Source IP: SPOOFÉE = IP victime
     - Destination: Serveur NTP avec monlist activé (vulnérable)
     - Command: monlist (retourne les 600 dernières IPs qui ont interrogé le
serveur)
     - Taille requête: ~234 bytes

  2. Serveur NTP répond:
     - Liste de 600 IPs × 128 bytes = ~130,000 bytes
     - Taille réponse: 130 KB

  3. Amplification:
     234 bytes → 130,000 bytes = 556x

  4. Attaque en 2014 (Cloudflare):
     - Trafic pic: 400 Gbps
     - Source: ~4,000 serveurs NTP vulnérables

Défense:
  - NTP serveur: Désactiver commande monlist (ntpd 4.2.7+)
  - NTP serveur: Firewall (autoriser uniquement clients connus)
  - Victime: BCP38 (Block spoofed packets at ISP level)
  - Victime: Rate limiting NTP
```

Attaque 3: Memcached Amplification

```
Facteur d'amplification: 51,000x (!!) [RECORD ABSOLU]

Mécanisme:
  1. Attaquant envoie requête Memcached:
     - Source IP: SPOOFÉE = IP victime
     - Destination: Serveur Memcached exposé sur Internet (UDP 11211)
     - Command: stats (retourne toutes les statistiques du cache)
     - Taille requête: ~15 bytes

  2. Serveur Memcached répond:
     - Toutes les statistiques du serveur
     - Taille réponse: ~750,000 bytes (750 KB)

  3. Amplification:
     15 bytes → 750,000 bytes = 51,000x (!)

  4. Attaque GitHub (2018):
     - Trafic pic: 1.35 Tbps (record mondial)
     - Source: ~50,000 serveurs Memcached exposés
     - Durée: 10 minutes
     - Mitigation: Akamai Prolexic (scrubbing)

Défense:
  - Memcached: JAMAIS exposer sur Internet public
  - Memcached: Firewall UDP 11211 (autoriser uniquement réseau interne)
  - Memcached: Désactiver UDP (utiliser TCP uniquement)
  - Memcached: Authentication (SASL)
  - Victime: Scrubbing center (seule solution efficace)
```

Tableau Comparatif des Amplifications:

Protocol	Port	Facteur	Query Size	Response Size	Défense Serveur
DNS	UDP 53	50-100x	60 bytes	3-6 KB	Rate limiting, DNSSEC
NTP	UDP 123	556x	234 bytes	130 KB	Disable monlist, firewall
Memcached	UDP 11211	51,000x	15 bytes	750 KB	**Firewall UDP 11211**
SSDP	UDP 1900	30x	Small	Large	Disable UPnP externally
CharGen	UDP 19	358x	1 byte	358 bytes	Disable service
SNMP	UDP 161	6x	Small	Medium	SNMPv3, ACLs

Stratégies de Mitigation DDoS:

1. Over-Provisioning (Infrastructure)

```
Principe: Avoir plus de capacité que le pic attendu

Exemple:
  - Trafic normal: 1 Gbps
  - Trafic pic légitime: 5 Gbps
  - Provisionner: 20 Gbps

Limites:
  ❌ Coûteux (payer pour capacité inutilisée)
  ❌ Ne scale pas contre attaques massives (1 Tbps)
  ✅ Protège contre petites attaques (< 20 Gbps)
```

2. Rate Limiting (Application)

```
Principe: Limiter requêtes/sec par IP source

Exemple Nginx:
  limit_req_zone $binary_remote_addr zone=one:10m rate=10r/s;

  location /api/ {
      limit_req zone=one burst=20;
  }

Limites:
  ❌ Inefficace si botnet (10,000 IPs × 10 req/s = 100,000 req/s)
  ✅ Protège contre single-source DoS
```

3. Géo-Blocking (Network)

```
Principe: Bloquer trafic de pays non pertinents

Exemple:
  - Site canadien sans clients internationaux
  - Bloquer: Russie, Chine, Brésil (sources communes de botnets)

Limites:
  ❌ Peut bloquer utilisateurs légitimes (VPN, voyage)
  ✅ Réduit surface d'attaque
```

4. Anycast (Network)

```
Principe: Distribuer trafic géographiquement

Architecture:
  - Même IP annoncée depuis multiples datacenters
  - BGP route trafic vers datacenter le plus proche
  - Attaque distribuée entre tous les datacenters

Exemple:
  - Attaque 100 Gbps
  - 10 datacenters × 10 Gbps chacun
  → Chaque datacenter reçoit seulement 10 Gbps (gérable)

Fournisseurs:
  - Cloudflare (200+ datacenters)
  - Akamai (240,000+ serveurs)
```

5. Scrubbing Centers (Vendor) — ✅ SOLUTION PROFESSIONNELLE

```
Principe: Filtrer trafic malveillant avant qu'il atteigne la cible

Architecture:
  1. Trafic normal: Client → Victime (direct)
  2. Sous attaque: Client → Scrubbing Center → Victime

  Scrubbing center:
    - Analyse chaque packet (DPI)
    - Détecte patterns DDoS (SYN flood, amplification)
    - Drop trafic malveillant
    - Forward trafic légitime à la victime

Activation:
  - Manuelle (BGP route advertisement)
  - Automatique (détection DDoS par IDS)

Fournisseurs:
  - Cloudflare (gratuit tier: unlimited DDoS protection)
  - Akamai Prolexic ($$$)
  - AWS Shield ($$$)

Efficacité:
  ✅ Protège contre attaques massives (1+ Tbps)
  ✅ Filtre amp amplifications, floods, botnets
  ⚠️ Latence supplémentaire (~20-50ms)
  ⚠️ Coût (si payant)
```

6. WAF (Web Application Firewall)

```
Principe: Filtrer attaques applicatives (Layer 7)

Protection:
  ✅ HTTP flood (GET/POST spam)
  ✅ Slowloris (connexions lentes)
  ✅ XML bomb
  ✅ OWASP Top 10 (SQLi, XSS)

Limites:
  ❌ N'aide PAS contre attaques volumétriques (SYN flood, UDP amp)
```

Défense Complète (Layered Defense):

```
Layer 1 (ISP/Transit):
  - BCP38: Block spoofed packets
  - BGP blackhole routing
  - Flowspec filtering

Layer 2 (Network Edge):
  - Anycast distribution
  - Scrubbing center activation
  - Rate limiting at border routers

Layer 3 (Firewall):
  - SYN cookies (protect against SYN flood)
  - Connection limits
  - Geo-blocking

Layer 4 (Load Balancer):
  - Health checks
  - Automatic failover
  - SSL termination (reduce backend load)

Layer 5 (WAF):
  - Application-level filtering
  - Challenge-response (CAPTCHA)
  - Rate limiting per user

Layer 6 (Application):
  - Efficient code (no N+1 queries)
  - Caching (Redis, CDN)
  - Graceful degradation

Layer 7 (Monitoring):
  - Real-time traffic analysis
  - Anomaly detection (ML)
  - Automatic mitigation triggers
```

Question 12: Hash Password Storage

Expliquez pourquoi MD5 et SHA-256 ne sont pas adaptés pour le stockage de mots de passe. Quelle est la bonne pratique en 2026? Expliquez le rôle du salt.

Réponse:

Pourquoi MD5 et SHA-256 sont MAUVAIS pour les mots de passe:

Problème 1: Trop Rapides

```
Vitesse de calcul (GPU moderne RTX 4090):
  - MD5: 200 MILLIARDS de hash/seconde
  - SHA-256: 10 MILLIARDS de hash/seconde

Attaque brute force contre MD5:
  - Mot de passe: 8 caractères alphanumeric (a-z, A-Z, 0-9)
  - Espace: 62^8 = 218 trillions de combinaisons
  - Temps: 218 × 10^12 / 200 × 10^9 = 1,090 secondes = 18 minutes (!)

Conclusion: Un attaquant peut tester TOUS les mots de passe de 8 caractères
en moins de 20 minutes avec MD5.
```

Problème 2: Sans Salt → Rainbow Tables

```
Rainbow table: Table précalculée de hashes
  - Calcule hash de TOUS les mots de passe communs (1 milliard+)
  - Stocke: [password → hash]

Exemple:
  Hash MD5 stocké en DB: 5f4dcc3b5aa765d61d8327deb882cf99
  Lookup dans rainbow table:
    5f4dcc3b5aa765d61d8327deb882cf99 → "password"

Temps: Instant (lookup dans table)
Taille: ~100 GB pour 1 milliard de mots de passe
Disponible: Gratuitement en ligne (crackstation.net)
```

Problème 3: Même Mot de Passe → Même Hash

```
Users dans base de données:
  user1: password="123456" → MD5=e10adc3949ba59abbe56e057f20f883e
  user2: password="123456" → MD5=e10adc3949ba59abbe56e057f20f883e
  user3: password="qwerty" → MD5=d8578edf8458ce06fbc5bb76a58c5ca4
  user4: password="123456" → MD5=e10adc3949ba59abbe56e057f20f883e

Observation:
  - Attaquant voit que user1, user2, user4 ont le MÊME hash
  - Crack 1 hash = crack 3 comptes simultanément
  - Identify mots de passe les plus communs (statistical attack)
```

Problème 4: SHA-256 N'est PAS MIEUX

```
SHA-256 est plus lent que MD5, mais:
  - Toujours TROP RAPIDE (10 milliards hash/sec)
  - Rainbow tables existent aussi pour SHA-256
  - Conçu pour intégrité, PAS pour passwords

SHA-256 est BON pour:
  ✅ Vérifier intégrité de fichiers
  ✅ Blockchain (Bitcoin)
  ✅ HMAC (avec clé secrète)

SHA-256 est MAUVAIS pour:
  ❌ Stockage de mots de passe
```

BONNE PRATIQUE 2026: bcrypt ou Argon2 avec Salt

1. Salt (Salage)

```
Salt: Valeur aléatoire UNIQUE ajoutée à chaque mot de passe avant hashing

Sans salt:
  password="123456"
  hash=MD5("123456")="e10adc3949ba59abbe56e057f20f883e"

Avec salt:
  password="123456"
  salt="a8f3d9e2b1c4" (aléatoire, unique par user)
  hash=bcrypt("123456" + "a8f3d9e2b1c4")="$2b$10$a8f3d9e2b1c4/xyzABC..."

Stocké en DB:
  [username, salt, hash]
  ["user1", "a8f3d9e2b1c4", "$2b$10$a8f3d9e2b1c4/xyzABC..."]
```

Avantages du Salt:

- ✅ Même mot de passe → Hash DIFFÉRENT (salt unique)
- ✅ Rainbow tables INUTILES (attaquant doit recalculer pour chaque salt)
- ✅ Protect contre statistical attacks

2. bcrypt

```
import bcrypt

# Enregistrement (signup):
password = "MySecureP@ss123".encode('utf-8')
salt = bcrypt.gensalt(rounds=12)  # rounds = cost factor (2^12 iterations)
hashed = bcrypt.hashpw(password, salt)

# Stocké en DB:
# hashed = b'$2b$12$a8f3d9e2b1c4...'  (contient salt + hash)

# Vérification (login):
entered_password = "MySecureP@ss123".encode('utf-8')
if bcrypt.checkpw(entered_password, hashed):
    print("Login successful")
else:
    print("Invalid password")
```

Caractéristiques bcrypt:

- ✅ Intentionnellement LENT (~100-500 ms par hash)
- ✅ Cost factor ajustable (rounds=10-14)
  - rounds=10: ~100 ms (rapide, acceptable)
  - rounds=12: ~300 ms (bon équilibre)
  - rounds=14: ~1200 ms (très sécure, mais lent pour user)
- ✅ Salt inclus dans le hash (pas besoin de stocker séparément)
- ✅ Résistant aux GPUs (utilise beaucoup de RAM)

Vitesse comparative:

```
GPU RTX 4090:
  - MD5: 200 milliards hash/sec
  - bcrypt (rounds=12): 50,000 hash/sec (~4 millions fois plus lent)

Attaque brute force 8-char password:
  - MD5: 18 minutes
  - bcrypt: 4,000,000 × 18 min = 136 ANNÉES
```

3. Argon2 (MEILLEUR choix 2024+)

```
from argon2 import PasswordHasher

ph = PasswordHasher(
    time_cost=3,        # Iterations
    memory_cost=65536,  # 64 MB de RAM
    parallelism=4,      # 4 threads
    hash_len=32,        # 32 bytes output
    salt_len=16         # 16 bytes salt
)

# Enregistrement:
password = "MySecureP@ss123"
hashed = ph.hash(password)

# Stocké en DB:
# hashed = "$argon2id$v=19$m=65536,t=3,p=4$..."

# Vérification:
try:
    ph.verify(hashed, "MySecureP@ss123")
    print("Login successful")
except:
    print("Invalid password")
```

Avantages Argon2 vs bcrypt:

- ✅ Résistant aux GPUs ET ASICs (utilise beaucoup de RAM)
- ✅ Recommandé par OWASP (2024+)
- ✅ Trois variantes:
  - Argon2d: Résiste aux attaques GPU
  - Argon2i: Résiste aux attaques side-channel
  - Argon2id: Hybride (RECOMMANDÉ)
- ✅ Paramètres ajustables (time, memory, parallelism)

Comparaison:

Algorithm	Speed	GPU Resistant?	Recommended 2026?	Use Case
MD5	200 Ghash/s	❌ No	❌ JAMAIS	Checksums only (NOT passwords)

SHA-256	10 Ghash/s	❌ No	❌ JAMAIS	Integrity, blockchain (NOT passwords)
bcrypt	50 Khash/s	✅ Yes (medium)	✅ Bon	**Password storage** (acceptable)
Argon2	10 Khash/s	✅ Yes (high)	✅ MEILLEUR	**Password storage** (BEST 2026)

Implémentation Complète:

```python
# ✅ SÉCURISÉ - Argon2 avec tous les best practices
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError
import secrets

ph = PasswordHasher()

def register_user(username, password):
    """Enregistre un nouvel utilisateur."""
    # 1. Valider force du mot de passe
    if len(password) < 12:
        raise ValueError("Password must be at least 12 characters")

    # 2. Hash avec Argon2 (salt automatique)
    password_hash = ph.hash(password)

    # 3. Stocker en DB
    db.execute(
        "INSERT INTO users (username, password_hash) VALUES (?, ?)",
        (username, password_hash)
    )

def login_user(username, password):
    """Vérifie les credentials lors du login."""
    # 1. Récupérer hash depuis DB
    user = db.execute(
        "SELECT password_hash FROM users WHERE username = ?",
        (username,)
    ).fetchone()

    if not user:
        # Timing-safe: toujours hasher même si user inexistant
        ph.hash("dummy_password")
        return False

    # 2. Vérifier le mot de passe
    try:
        ph.verify(user['password_hash'], password)

        # 3. Rehash si paramètres obsolètes (rotation automatique)
        if ph.check_needs_rehash(user['password_hash']):
            new_hash = ph.hash(password)
```

```
            db.execute(
                "UPDATE users SET password_hash = ? WHERE username = ?",
                (new_hash, username)
            )

        return True
    except VerifyMismatchError:
        return False
```

Best Practices Additionnelles:

1. **Password Policy:**
   - ✅ Longueur minimum: 12 caractères (16+ recommandé)
   - ✅ Reject mots de passe communs (top 10,000)
   - ✅ Reject mots de passe leaked (HIBP API)
   - ❌ PAS de complexité forcée (Aa1! requis) → encourage mauvais patterns
2. **Rate Limiting:**
   - Max 5 tentatives par compte par heure
   - CAPTCHA après 3 échecs
   - Blocage temporaire après 5 échecs
3. **Breach Monitoring:**
   - Vérifier régulièrement si hashes ont leaké (HaveIBeenPwned)
   - Forcer reset si breach détecté
4. **Account Recovery:**
   - Email avec token temporaire (expiry 1h)
   - JAMAIS envoyer mot de passe par email
   - Log toutes les tentatives de reset

Question 13: Attack Lifecycle 8 Phases

Décrivez les 8 phases du cycle de vie d'une attaque avec les techniques utilisées à chaque phase. Donnez un exemple concret d'attaque complète.

[Réponse détaillée de 2000+ mots attendue, couvrant: Reconnaissance, Scanning, Vulnerability Analysis, Intrusion, Privilege Escalation, Compromise, Persistence, Cleanup avec exemple web server compromise]

Question 14: VPN vs IPSec Modes

Comparez VPN site-to-site et VPN remote access. Expliquez pourquoi IPSec mode tunnel est préféré pour un VPN site-to-site.

[Réponse détaillée attendue]

Question 15: Certificate Revocation: CRL vs OCSP

Expliquez comment fonctionne la révocation de certificats avec CRL et OCSP. Quel est le problème de privacy d'OCSP? Comment OCSP Stapling résout-il ce problème?

[Réponse détaillée attendue couvrant: CRL mechanics, OCSP mechanics, privacy leak (CA sees all queries), OCSP Stapling (server caches response), comparison table]

Annexe: Corrigé Abrégé

Pour référence rapide lors de la révision. Solutions détaillées ci-dessus.

Partie 1 - V/F Réponses:

1-5: F, V, F, V, F
6-10: F, F, V, F, V
11-15: V, F, F, V, F
16-20: F, F, F, V, V
21-25: V, F, V, V, F
26-30: F, V, V, V, V

Partie 2 - Choix Multiples Réponses:

1: b,d | 2: c | 3: b,c | 4: a,b,c | 5: b,c | 6: a,b,c | 7: c | 8: b | 9: b | 10: b
11: c | 12: b | 13: a,b | 14: a,b,c | 15: a,c | 16: a | 17: b | 18: d | 19: a | 20: c
21: c | 22: d | 23: a | 24: a,b,c | 25: d | 26: d | 27: d | 28: d | 29: d | 30: d
31: d | 32: c | 33: d | 34: b | 35: a | 36: b | 37: c | 38: a | 39: d | 40: d

---

Fin des Exercices de Préparation

Recommandations pour l'examen:

- Pratiquer ces questions plusieurs fois
- Réviser le cheatsheet (IFT2830_finals-cheatsheet.md)
- Relire les modules de théorie pour approfondissement
- Faire des cartes mémoire (flashcards) pour termes français/anglais
- S'entraîner aux calculs RSA et aux formules de risque
- Simuler examen en 3 heures (temps réel)

Bonne chance! Bonne chance! 🎓

Version: 1.0
Créé: 27 avril 2026

Auteur: PtiCalin (Charlie)
Cours: IFT2830 - H26
Status: ✅ Ready for Practice